



BLUE BITS

البرمجة التفرعية

نظري (6)

ملف داعم

2026/2025

السنة الرابعة



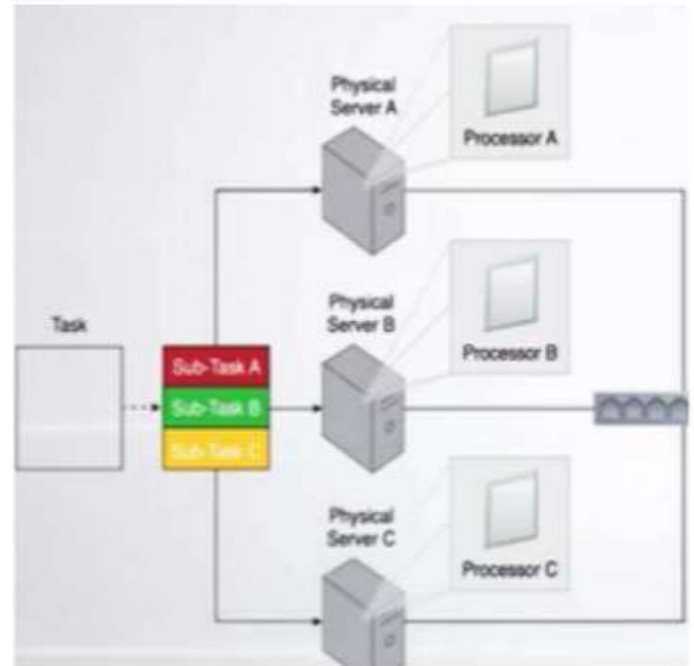
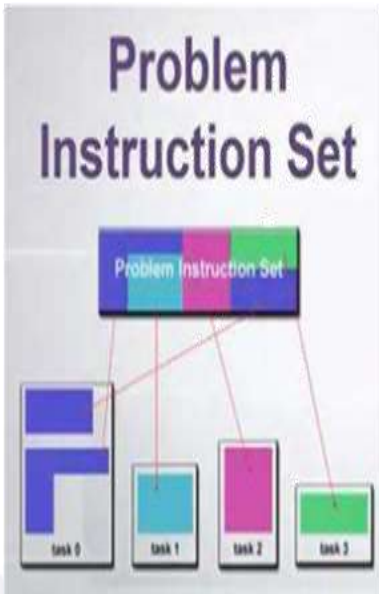
مكتبة شهاب
الشهاب الجديدة - يثرب - يثرب - يثرب

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com

- يجب معرفة أنه ليس بالضرورة أن يؤدي العمل التفرعي إلى التحسين المتوقع.
- ليكن لدينا مشكلة برمجية تجزأ إلى أجزاء كل جزء هو بمثابة *task* وبعد تجزئته، يتم معالجة كل جزء من قبل *process* محددة.

فهل هذا قابل للتطبيق في كافة القضايا البرمجية؟



سؤال: أي من العمليات التالية قابلة للتنفيذ التفرعي؟

1. $Z = X + Y$

$$\begin{bmatrix} Z1 & Z2 \\ Z3 & Z4 \end{bmatrix} = \begin{bmatrix} X1 & X2 \\ X3 & X4 \end{bmatrix} + \begin{bmatrix} Y1 & Y2 \\ Y3 & Y4 \end{bmatrix}$$

جمع العناصر المتقابلة

فهل هذه العملية قابل للإسناد على التفرع؟

نعم، كل *process* تأخذ عنصر من الأول وتجمعه مع عنصر من الثاني

2. $Z = X * Y$

$$\begin{bmatrix} Z1 & Z2 \\ Z3 & Z4 \end{bmatrix} = \begin{bmatrix} X1 & X2 \\ X3 & X4 \end{bmatrix} * \begin{bmatrix} Y1 & Y2 \\ Y3 & Y4 \end{bmatrix}$$

هل هذه العملية قابلة للإسناد على التفرع؟

نعم يمكن ذلك في حال أسندنا كل عنصر من المصفوفة *Z* إلى *process* مختلفة.



$Z_1 = X_1Y_1 + X_2Y_3$ نرسلها إلى *Process1*

$Z_1 = X_1Y_2 + X_2Y_4$ نرسلها إلى *Process2*

وهكذا حتى تكتمل جميع عناصر Z

وبالتالي أصبح لدينا استقلالية في العمل أي قادرين على العمل على التوازي

$$Z = X * Y + Z \quad 3.$$

هنا يتوجب علينا تقسيم العمل الى مرحلتين بحيث لا يمكن أن تقوم الجمع قيمة Z مع ناتج عملية الضرب $(X * Y)$ على التوازي.

لا بد من انتهاء عملية الضرب $(X * Y)$ أولا ومن ثم نقوم بعملية الجمع مع Z للحصول على النتيجة النهائية.

Performance Metrics for Parallel Systems المعايير الخاصة في النظم التفرعية

1. *Execution Time* زمن التنفيذ

2. *Total Parallel Overhead* العبء الكلي للعمل التفرعي

3. *SpeedUp* السرعة

4. *Efficiency* الكفاءة

5. *Cost* التكلفة

لتقييم أي عمل برمجي أي هل أنا قادر على تنفيذ هذا العمل بشكل سليم 100% فيجب الأخذ بعين الاعتبار هذه المعايير الخمسة وقياسها يعتمد على قيم رقمية

(1) Execution Time

بعض القضايا البرمجية تقتضي انتظار جزء من العمل ليتم تنفيذه بشكل تسلسلي بحث ولا يوجد فائدة من وجود أكثر من

Process واحدة للتنفيذ

$O(n)$ (الزمن التسلسلي) *Ts: Serial Time*

$O(\log n)$ (الزمن التفرعي) *Tp: Parallel Time*

$$Z = \underbrace{x * y}_{Tp} + \underbrace{z}_{Ts}$$

مثلاً:

$$Tp \quad Ts$$

(2) Total Parallel Overhead

هو إجمالي الوقت المستغرق في حل مشكلة تم جمعها وفق جميع عناصر المعالجة وهو PTp

Ts : هو الوقت المستغرق في أداء عمل مفيد وباقي الناتج هو العبء $T0$

$$T0 = PTp - Ts$$

overhead يسمى بالزمن الضائع

لدينا عدد p من ال *processes* في جزء تفرعي، فيكون الزمن المطلوب لتنفيذ العمل على التفرع

$$Total\ time = P\ Tp$$

$$overhead = P\ Tp - Ts$$

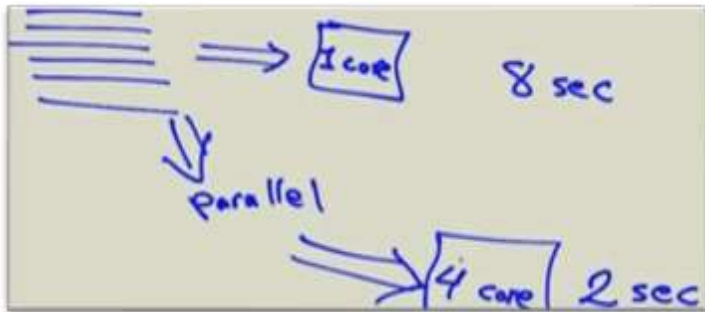
ليكن لدينا 4 معالجات في مصفوفة ثنائية أي: $4 * T_p$

1 $S_{SpeedUp}$:

هو نسبة الوقت المستغرق لحل مشكلة على عنصر معالجة واحد (باستخدام أفضل خوارزمية تسلسلية) إلى الوقت اللازم لحل نفس المشكلة على جهاز كمبيوتر متوازي مع عدد P من عناصر المعالجة المتطابقة:

$$S = T_s / T_p$$

ليكن لدينا:



زمن تسلسلي: 8 Sec

الزمن الإجمالي لإنجاز العمل على التفرع: 2Sec

$$S = T_s / T_p = 8 / 2 = 4 \text{ sec}$$

$$S = T_s / T_p = o(n) / o(\log n) = o(n / \log n)$$

3 $E_{Efficiency}$:

الكفاءة هي مقياس لجزء من الوقت الذي يتم فيه استخدام عنصر المعالجة بشكل مفيد في النظام المتوازي المثالي: السرعة تساوي P والكفاءة تساوي 1 أما في التطبيق العملي السرعة أقل من P والكفاءة بين الصفر والواحد

$$E = S / P$$

$$E = S / P = o(n / \log n) / n = o(1 / \log n)$$

4 $Cost$:

التكلفة هي مجموع الوقت الذي يقضيه كل عنصر معالجة في حل المشكلة

$$Cost = P T_p$$

مثال:

مهمة معينة يتم تنفيذها على التسلسل في 6 sec

عندما يتم تنفيذ نفس المهمة على نظام مكون من 5 معالجات يتم تحقيق زيادة في السرعة قدرها 3

أوجد $Cost, Efficiency, Overhead$

الحل:

$$T_s = 6, p = 5, S = 3$$

$$S = T_s / T_p \Rightarrow T_p = 2, T_0 = P * T_p - T_s = 5 * 2 - 6 = 4$$

:Overhead

$$E = S / P = 3 / 5 = 0.6 * 100 = 60 \%$$

:Efficiency

$$Cost = P * T_p \Rightarrow Cost = 5 * 2 = 10$$

:Cost

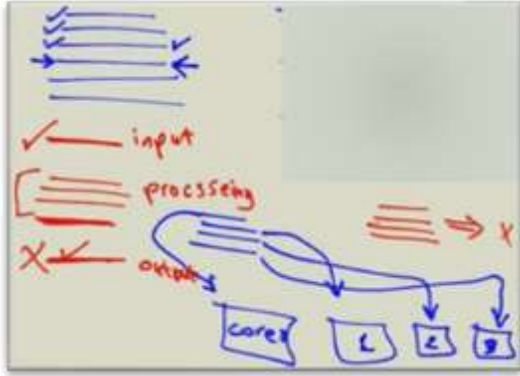
التحديات التي تواجه العمل التفرعي $Challenges of Parallel Programming$:

1- $Serial Code$: بعض الخطوات تحتاج معالجة تسلسلية

2- $Overhead Parallelization$: عبارة عن التزامنية

المفروضة

(تحميل زائد لعملية التزامنية بسبب عملية التوزيع والانتقال)



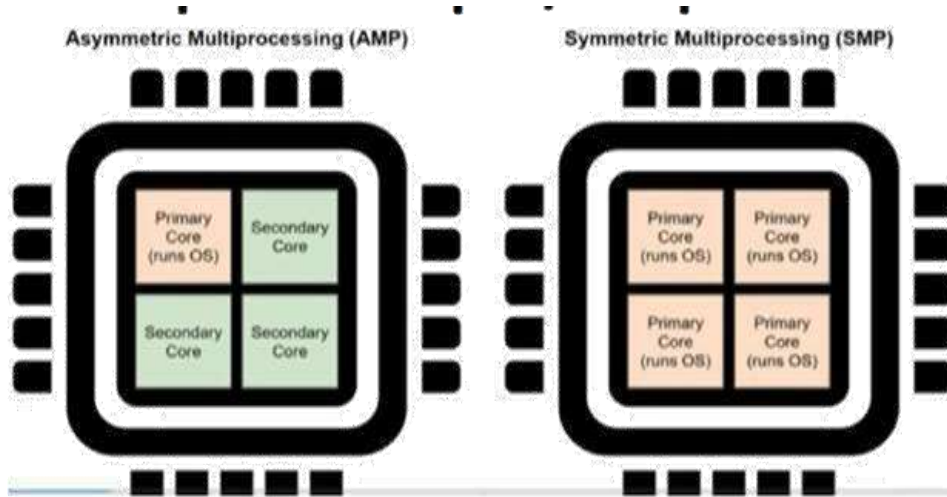
في الأساس عملية توزيع الـ *Tasks* للعمليات تأخذ زمن، وعملية إعادة (استرجاع) البيانات بعد معالجتها والقيام بالعمليات الحسابية عند كل *processor* تأخذ زمن.

-3 Synchronization:

تزامنية بين الـ *processes* مع بعضها البعض في حين هذا لم نكن نفكر به حين ننفذ البرمجة العادية

-4 Sequential input/output

هنا يجب أن ننتظر ليتم إدخال بيانات معينة من المستخدم وهذا الوقت ضائع علينا فقد يتم إدخال بعض طرحها كـ *output* المعلومات ويجب أن نعالجها قبل أن يتم (مثلاً إصدارها على الشاشة أو طباعتها على الورقة هذا الشيء لم نعالجه حالياً) - لدينا نوعين من المعالجات:



مثلاً:

لدينا معالج واحد *SMP* بعدد من الـ *cores* المختلفين فعندما نريد قياس العمل التفرعي نحتاج إلى كود قابل للتفرع بالإضافة إلى *Hardware* تدعم العمل التفرعي

البرمجة التفرعية تواجه عدد من المشاكل منها:

إن التفرعية لا تؤدي دائماً إلى زيادة في سرعة وكفاءة التنفيذ فمثلاً إذا كان لدينا قضية ضخمة تحتاج للتفرعية فأول مشكلة نواجهها

فنقترح عدد عشوائي من الـ *processes* لتوزيع العمل. علماً أنه يوجد عدد من المعالجات إذا قمنا كمبرمجين هي عدد الـ *processes*

بتجاوزه تتلاشى العملية التفرعية أي يوجد عدد أمثل للمعالجات نعتمده في آلية التقسيم

مثلاً:

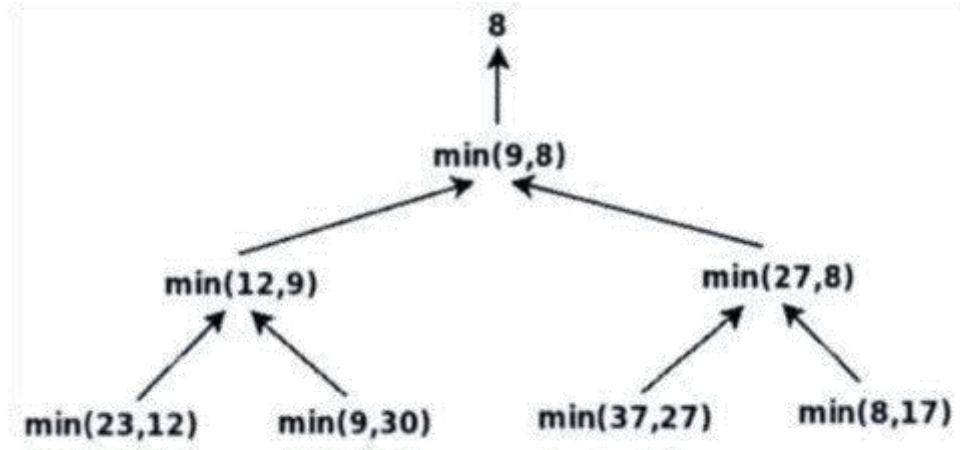
ليكن لدينا 8 قيم موجودة ضمن مصفوفة



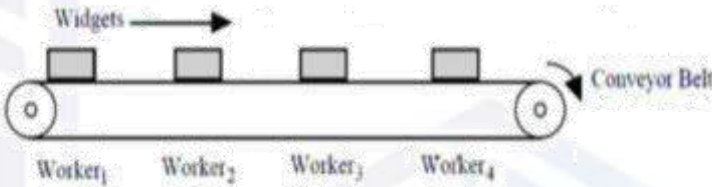
والمطلوب *Find the minimum element*

عدد المهام: $n = 4$, $[23,12,9,30,37,27,8,17]$

الحل:



دائماً سوف نفكر ما هو العدد والحجم المناسب للمهام الذي يمكننا تقسيم العمل من خلاله ويحقق الهدف من التفرعية دون أن أفقد أهمية العمل التفرعي
فهنا نستطيع بناء *graph* (بنية شجرية) ونسند لكل *process* قيمتين وكل واحد يرسل للثاني القيمة الأصغر



(الدقة): *Granularity*

عدد وحجم المهام التي تتوزع على المشكلة هو الذي يحدد الدقة

- ليكن لدينا عمل وهذا العمل يتضمن 10 صناديق وعملية معالجة كل صندوق تتم من خلال 4 مراحل

فالصندوق سوف يمر ب 4 عمال (*worker*) وكل عامل ينجز مرحلة إذا كان العمل تفرعي ، أما إذا كان تسلسلي فسيكون هناك عامل واحد ينجز جميع المهام)

في العمل التسلسلي العامل الواحد ينجز ال 4 مراحل على التسلسل، أي كل صندوق سوف يمر بكل مرحلة ويأخذ وقته وصولاً للنهاية، علماً أنّ الزمن اللازم لإنجاز كل مرحلة متساوي في كل مرحلة وهو T

speed up

$$= \frac{\text{الزمن التسلسلي}}{\text{الزمن التفرعي}}$$

$$\text{speedup} = \frac{\text{the time for the sequential case}}{\text{the time for the parallel case}}$$

فإذا كانت واحدة القياس للزمن التسلسلي مثلاً ساعة (*onepoint*)

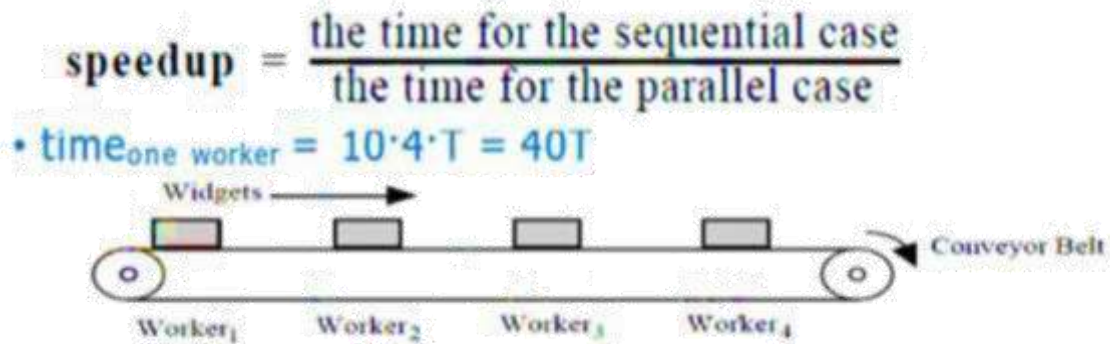
وكان العمل سوف ينفذ على 4 مراحل فهو سيستغرق $4 * 10 = 40$

حيث تدل 10 على عدد الصناديق

زمن إنجاز العامل الواحد (في العمل التسلسلي)

$$worker = 10 * 4 T = 40 T$$

حيث: تدل T على الوقت اللازم لإنجاز كل عملية



الوقت اللازم لتنفيذ العمل التفرعي (عند تقسيم ال 4 مراحل على 4 عمال يعمل كل منهم بشكل مستقل عن الآخر)

$$Time \text{ assembly line} = 4 \cdot T + (10 - 1) T = 4T + 9T = 13T$$

أول عبوة تعمل بشكل كامل تمر على كل مرحلة بحيث لا يوجد عبوة قبلها مكان مرحلة ما لذلك تأخذ الوقت الكامل لكل مرحلة (الزمن اللازم لكل مرحلة T ولدينا 4 مراحل -» $4T$) أي سوف تعمل على التسلسل

العبوات المتبقية هي 9، وستعمل على التفرع حيث ستأخذ الوقت T

هي قيمة رقمية تعطينا معيار زيادة السرعة في حال نقّذنا البرنامج على التفرع بدلا من التسلسل نلاحظ ب $Speedup \text{ (assembly line)}$

$$speedup_{\text{assembly line}} = \frac{\text{time for one worker}}{\text{time for assembly line}} = \frac{40 \cdot T}{13 \cdot T} = 3.07$$

أي وفرت 3 أضعاف الزمن

: Amdahl's Law

يفيد في عملية تحديد سرعة الزمن فيما إذا كان البرنامج تم تنفيذه على التسلسل مقارنة فيما إذا البرنامج تم تنفيذه على التفرع.

$speedup$: مقياس السرعة وهو مقياس كمي للأداء، يحدد فائدة تشغيل البرنامج بالتوازي - التسريع هو نسبة الوقت الذي يستغرقه تنفيذ برنامج بشكل تسلسلي (باستخدام معالج واحد) إلى الوقت الذي يستغرقه التنفيذ بالتوازي (باستخدام العديد من المعالجات)

- The formula for speedup: $S(n) = \frac{T(1)}{T(n)}$ = $\frac{\text{Time taken to execute the program using single processor}}{\text{Time taken to execute the program using n no. of processors}}$
- For example: $T(1) = 1 \text{ sec}$ & if $n=2$ processors then $T(2) = \frac{1}{2} = 0.5 \text{ sec}$
Hence $S(n) = T(1)/T(2) = 1/0.5 = 2$ means Speedup increases by 2 times

أي أن السرعة تضاعفت عند استخدام *2 processors* ووفرت نصف الزمن المستهلك للقيام بالمهام

Amdahl's Law : هو أحد قوانين تسريع الأداء

- يعتمد على حجم مشكلة ثابت (أو عبء عمل ثابت)
- في الواقع، للحفاظ على كفاءة النظام ثابتة *Efficiency* علينا زيادة كل من حجم المشكلة وعدد المعالجات في وقت واحد
- ينص قانون *Amdahl's* على أنه بالنسبة لحجم مشكلة معين، لا تزداد السرعة خطياً مع زيادة عدد المعالجات، بالواقع يصبح *speedup* مشبعاً أكثر.
- وينص أيضاً على أن جزءاً صغيراً من البرنامج الذي لا يمكن جعله تفرعياً (الجزء التسلسلي) سيحد من السرعة الإجمالية المتاحة من التفرع
- عادة أي مشكلة رياضية أو هندسية كبيرة تتكون من عدة أجزاء قابلة للتفرع وعدة أجزاء متسلسلة

$$Computation Problem = Serial Part + Parallel Part$$

غالباً عملية قياس الزمن المطلوب لإنجاز عمل ما يساوي الزمن المطلوب لإنجاز الجزء التسلسلي + الزمن المطلوب لإنجاز الجزء التفرعي

percentage (percentage of parallel, percentage of serial):

- النسبة المئوية للكود الكلي فنحن نقترح نسبة مئوية مثلاً هذا قابل للتنفيذ تفرعياً بنسبة 100% ويمكن قابل للتنفيذ تفرعياً بنسبة 80%، و 20% على التسلسل

$$speed up = 1 / (1 - p + (p/N))$$

نسبة الجزء المنفذ على التفرع *P* , عدد المعالجات *N*

Single Core Vs Multi Core.

Percentage (Percentage of Parallel, Percentage of Serial).

Machine Type (?!!!!)

$$Speed up = 1 / (1 - p + (p/N));$$

N : number of core ; *p* نسبة الجزء المنفذ على التفرع القابل للتفرع

بفرض *core = 3* و *P = 0.75, N = 3* عندها يكون مقدار زيادة السرعة بتطبيق القانون هو الضعفين فيما لو نفذنا نفس العمل على *1 - core* فعلياً لدى الحساب لقيمة

$$Speed up = 1 / (1 - 0.75 + (0.75 / 3)) = 2$$

السرعة المتوقعة هي ثلاثة الأضعاف ولكن فعلياً نتج لدينا ضعفي السرعة فقط هل التفرعية في تنفيذ الشيفرة البرمجية يؤدي إلى تحسين الأداء دوماً؟

الحالة الأولى: إذا كان لدينا شيفرة برمجية قابلة للتفرع بنسبة 0.10 والباقي 90% قابل للتنفيذ على *core* واحدة ولدينا *N = 90 cores* عندها ينتج لدينا:

$$Speedup = 1.109$$

الحالة الثانية: تنفيذ 90% من البرنامج بشكل تفرعي على 10 *cores* ينتج لدينا: *Speedup = 5.2*

لدى تنفيذ الجزء الأكبر على التفرع أعطى كفاءة أفضل بكثير فيما إذا كان جزء التسلسل أكبر.



من مساوى هذه الطريقة عدم حساب *Overhead Parallelism* أي أنها تُراعي نسب تنفيذ الشيفرة دون مراعاة الفترات الزمنية للانتقال بين حالات التنفيذ المختلفة، لإيقاف التنفيذ التفرعي والانتقال إلى جزء تسلسلي ثم إيقاف ومتابعة العمل التفرعي.

مثال 1

70% من وقت تنفيذ البرنامج يحدث داخل حلقة يمكن تنفيذها بالتوازي، والباقي 30% تسلسلياً. ما أقصى سرعة متوقعة من نسخة متوازية من البرنامج تعمل على 8 وحدات معالجة مركزية؟
الحل:

$$P = 70\% = 0.7, \quad N = 8$$

$$Speed\ up = 1 / (1 - P + (P/N));$$

$$= 1 / (1 - 0.7 + (0.7/8)) = 2.580$$

مثال 2

80% من وقت تنفيذ البرنامج يحدث داخل حلقة يمكن تنفيذها بالتوازي، والباقي 20% تسلسلياً. ما أقصى سرعة متوقعة من نسخة متوازية من البرنامج تعمل على 8 وحدات معالجة مركزية
 $Speed\ Up = 1 / (1 - 0.8 + (0.8/8)) = 3.333$

Gustafson's Law

$$Speed\ up = [(S + N(1 - S)) / (S + (1 - S))] - On$$

S : النسبة المئوية للتسلسل

N : عدد النوى

$(1 - S)$: النسبة المئوية للتوازي

On : *Overhead Time*

$$S = 0.25, N = 3, On = 0.8$$

$$Speedup = [(0.25 + 3(0.75)) / (0.25 + (0.75))] - 0.8 = 1.7 == 2$$

مثال:



Sorting Algorithms

الترتيب باستخدام المبدأ التفرعي:

إعادة ترتيب قائمة من الأرقام بترتيب تصاعدي (غير تنازلي تماماً)

Potential speedup التسريع المحتمل:

$O(n \log n)$ مثالي لأي خوارزمية فرز تسلسلية دون استخدام خصائص خاصة للأرقام

أفضل ما يمكن أن نتوقعه بناءً على خوارزمية الفرز التسلسلي باستخدام n من المعالجات - التعقيد المثالي للزمن التفرعي:

$$\frac{o(n \log n)}{n} = o(\log n)$$

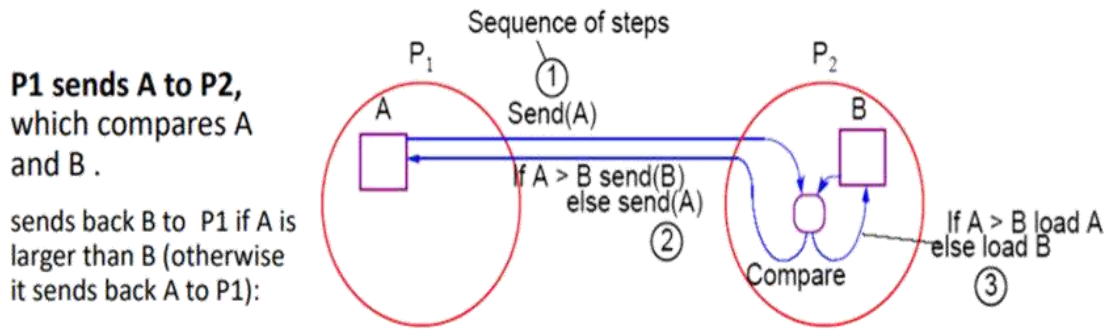
أي خوارزمية الترتيب ستعمل على المبدأ التفرعي ويكون تعقيدها $o(\log n)$

Message – Passing Compare and Exchange خوارزميات مقترحة للمقارنة والتبديل:

classical sequential sorting algorithms :

Two numbers, say A and B , are compared. If $A > B$, A and B are exchanged, i.e.:

if ($A > B$) { temp = A; A = B; B = temp; }



شرح:

ترسل $P1$ القيمة التي لديها إلى $P2$ تقوم $P2$ بالمقارنة إذا كانت القيمة الموجودة في $P1$ أكبر من القيمة الموجودة في $P2$ ترسل القيمة الموجودة في $P2$ إلى $P1$ وتبقى القيمة الأكبر لديها، أما إذا كان العكس ترسل القيمة التي جاءتها من $P1$ إلى $P1$ وتبقى القيمة الأكبر لديها

Version 1: Code:

```

1 Process P1
2 send (&A, P2);
3 recv (&A, P2);
4 Process P2
5 recv(&A, P1); if (A > B) {
6 send (&B, P1);
7 B = A;}
8 else
send (&A, P1);

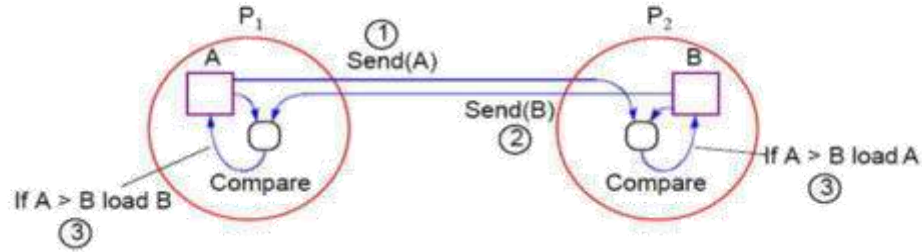
```

مشكلة هذا الإصدار أن بعض المعالجات تعمل أكثر من المعالجات الأخرى



Version 2:

- P1 sends A to P2 and P2 sends B to P1.
- Then both processes perform compare operations.
- P1 keeps the smaller of A and B and P2 keeps the larger of A and B:



شرح:

تأخذ P2 القيمة الأصغر و P1 كلاهما ترسل القيمة للطرف الآخر وكلاهما تقوم بعملية المقارنة ولكن القيمة الأكبر عند P2 و الأصغر عند P1 .

Version 2: Code:

```

1  Process P1
2  send (&A , P2);
3  recv (&B , P2);
4  if (A > B) A = B;
5  Process P2
6  recv(&A, P1);
7  send (&B, P1);
8  if (A > B) B = A;
```

ملاحظة حول دقة الحسابات المكررة:

- تفترض التعليمات البرمجية السابقة أن شرط $A > B$ سيُرجع نفس الإجابة المنطقية في كلا المعالجين.
- يُمكن للمعالجات المختلفة التي تعمل بدقة مختلفة أن تنتج إجابات مختلفة إذا تمت مقارنة الأرقام الحقيقية.
- ينطبق هذا الموقف على أي مكان يتم فيه تكرار العمليات الحسابية في معالجات مختلفة لتقليل تمرير الرسائل

تقوم العملية P1 بإجراء `send()` أولاً وتقوم العملية P2 بتنفيذ الدالة `recv()` أولاً لتجنب حالة *deadlock* بدلاً من ذلك، يمكن P2 و P1 إجراء `send()` أولاً وذلك إذا كانت عملية الإرسال *locally blocking* (غير متزامنة) لكل من

مع ضمان وجود تخزين *buffer* مع العلم ان هذا الأسلوب لا يعتبر تمرير رسائل آمن *safe message – passing* مؤقت كافٍ

ملاحظة:

أهم شيء بالكود أنهما لا ينتظران بعضهما

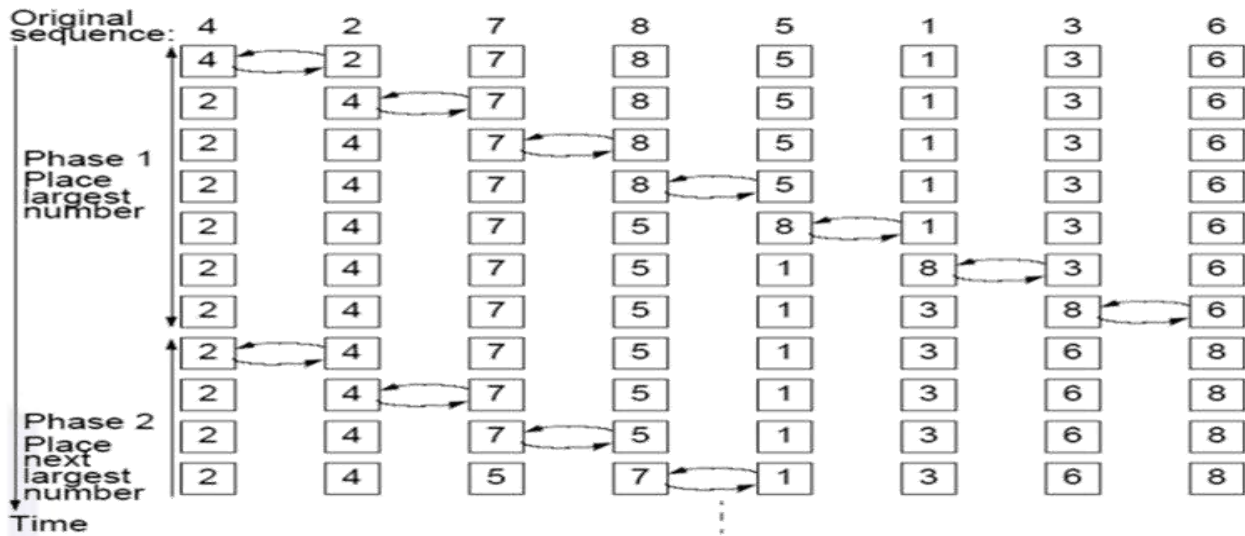
Bubble Sort

طريقة العمل: تقارن الخوارزمية كل عنصرين متجاورين في القائمة، فإذا كان العنصر الأول أكبر من الثاني، يتم تبديل أماكنهما.

النتيجة: بعد نهاية المرور الأول على القائمة بأكملها، يستقر أكبر رقم في أقصى اليمين (نهاية القائمة).

التكرار: تعود الخوارزمية لتبدأ من جديد، ولكنها تتجاهل الرقم الأخير الذي استقر في مكانه، وهكذا حتى تُرتب القائمة بالكامل.

مثال:



لدينا 8 قيم نقارن العمود الأول مع الثاني وهكذا .. ونضع الأكبر نحو نهاية السلسلة وعدد عمليات المقارنة بعدد العناصر بالمصفوفة.

- في كل مرة العدد الأكبر الذي سيوضع في نهاية السلسلة سوف يمر بعدد مقارنات $n - 1$

```

9 With numbers held in array a[]:
10 for (i = n - 1; i > 0; i --)
11 for (j = 0; j < i; j ++){
12 k = j + 1;
13 if (a[j] > a[k]) {
14 temp = a[j];
15 a[j] = a[k];
16 a[k] = temp; } }

```



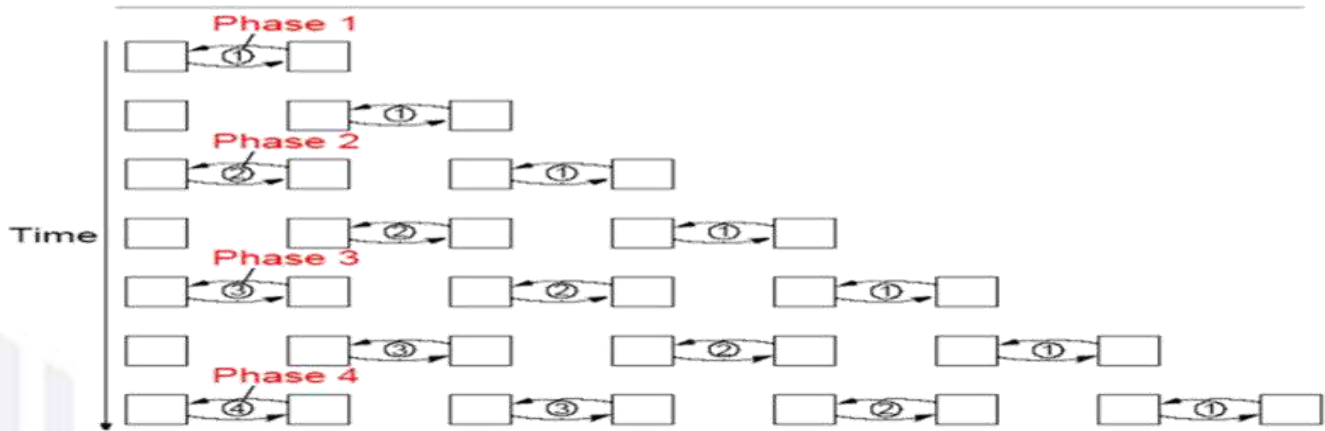

عدد عمليات المقارنة والتبادل:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

يشير إلى التعقيد الزمني $O(n^2)$ لأن عملية المقارنة والتبادل الفردية لها تعقيد ثابت $O(1)$

Parallel Bubble Sort

Iteration could start before previous iteration finished if does not overtake previous bubbling action:



يمكن أن يبدأ التكرار قبل انتهاء التكرار السابق إذا لم يتجاوز الإجراء الفقاعي السابق، من المحتمل أن يؤدي هذا التوازي إلى فرز أكثر كفاءة

Odd – Even (Transposition) Sort

يوجد اختلاف بينها وبين *Bubble sort*

يعمل على مرحلتين متناوبتين، المرحلة الزوجية والمرحلة الفردية

- **المرحلة الزوجية:** تتبادل العمليات ذات الأرقام الزوجية الأرقام مع العنصر (الجار) الأيمن

- **المرحلة الفردية:** تتبادل العمليات ذات الأرقام الفردية الأرقام مع العنصر (الجار) الأيمن

ملاحظة:

مرة نأخذ البداية فردية: $(P1 - P2)(P3 - P4)(P5 - P6)...$

مرة نأخذ البداية زوجية: $(P0 - P1)(P2 - P3)(P4 - P5)...$

ولدي 8 معالجات

Sorting eight numbers

Step	P ₀	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆	P ₇
0	4	2	7	8	5	1	3	6
1	2	4	7	8	1	5	3	6
2	2	4	7	1	8	3	5	6
3	2	4	1	7	3	8	5	6
4	2	1	4	3	7	5	8	6
5	1	2	3	4	5	7	6	8
6	1	2	3	4	5	6	7	8
7	1	2	3	4	5	6	7	8

ملاحظة:

الخطوات تكون على عدد الـ *processes*

Merge Sort

التقسيم (Divide): يتم تقسيم القائمة غير المرتبة إلى نصفين.

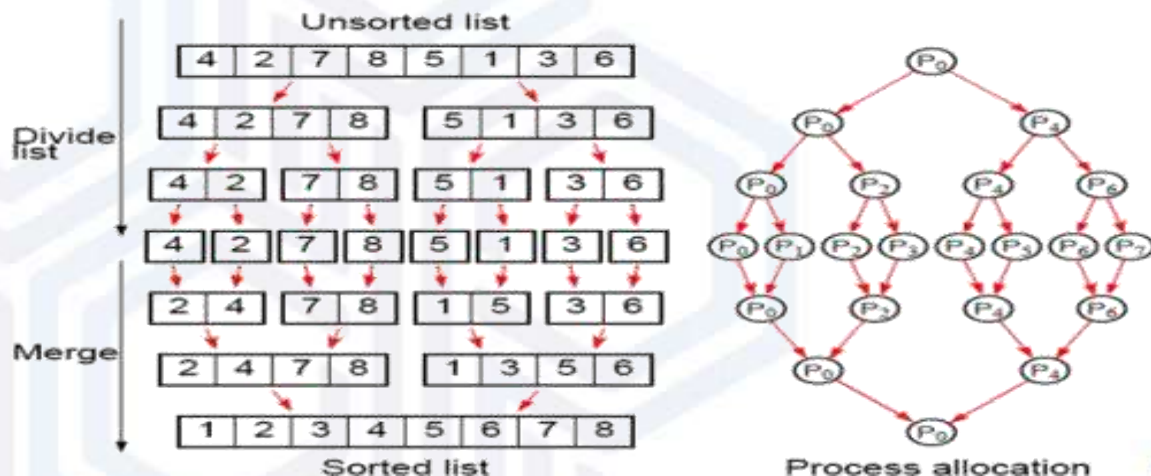
التكرار: يستمر تقسيم كل نصف إلى نصفين أصغر بشكل متكرر حتى نصل إلى أرقام منفردة (كل رقم بمفرده).

الدمج (Merge): تبدأ عملية الدمج بدمج كل رقمين معاً في قائمة صغيرة مرتبة مكونة من عنصرين.

توسيع القوائم: يتم دمج القوائم المكونة من عنصرين لتكوين قوائم مرتبة من 4 عناصر، ثم دمجها لتكوين قوائم من 8 عناصر، وهكذا.

النتيجة النهائية: تستمر هذه العملية حتى يتم دمج جميع العناصر في قائمة واحدة كبيرة ومرتبة بالكامل.

مثال:



التحليل *Analysis**Sequential :*التعقيد الزمني المتسلسل هو $O(n \log n)$ *Parallel:*

$2 \log n$ من الخطوات في الإصدار التفرعي ولكن يمكن أن تحتاج كل خطوة لإجراء أكثر من عملية أساسية واحدة، اعتماداً على عدد الأرقام التي تتم معالجتها

تذكرة:

الهدف من البرمجة التفرعية:

1. تحسين عمل الخوارزميات
2. توزيع المهام على عدة *Processes* بهدف توفير الوقت

"كل شيء في الحياة عليك انتظاره, إلا الأحلام .. تنتظرك كل يوم ان تخطو إليها"

2026/2025

السنة الرابعة



BLUE BITS



أعضاء الفريق



الفريق التقني



الفريق العلمي

إبراهيم هبرة	عائشة العُمر العلّوش	إيناس بيلوني	ظلال بابا
إسراء حاج حمدي	عمر قظان	تسنيم طالب	عائشة قلّه
بتول لبّان	فاطمة الحسين	حلا إبراهيم	عائشة مُصطفى
بشرى كوكش	قاسم يحيى	دلال تسقيه	قمر كمد
بنيامين حكيمان	كريست ياغجيان	ديانا صبّاغ	محمد يمان جلال
تسنيم حوكان	محمد منير	زهير حّقامي	محمود غزال
تسنيم نور الدّين	مديحة جّمّال	شكرية سُلطان	وفاء حّقامي
جميل هلال	ميرا فضلي	شهد كرّوم	يوسف نيّال
جودي جلال	نجم الدّين درويش		
خديجة بيازيد	هبة الله كتلو		
رشا معراوي	وداد قلعيّة		
سدرة مبيض	ياسر صبّاغ		

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

